

Programas, Máquinas e Computação

Exemplo de programa monolítico com instruções rotuladas:

```
1: faça F vá_para 2
2: se  $T_1$  então vá_para 1 senão vá_para 3
3: faça G vá_para 4
4: se  $T_2$  então vá_para 5 senão vá_para 1
```

figura 2.3 Instruções rotuladas

Exemplo de programa iterativo:

```
(se  $T_1$ 
  então enquanto  $T_2$ 
    faça (até  $T_3$ 
      faça (V;W))
  senão (✓))
```

figura 2.5 programa iterativo

Exemplo de programa recursivo:

Exemplo 2.6

Programa Recursivo.

```
P é R;Z onde
  R def F;(se T então R senão G;S),
  S def (se T então ✓ senão F;R)
  Z def (se T então G senão F)
```

figura 2.6 - Programa recursivo

Converter todos os exercícios para Programas Monolíticos com Fluxograma, com instrução rotulada. Programa iterativo.

Programa recursivo. Alguns exercícios tem exemplos para orientar a construção da solução.

8)

Desenvolver um programa monolítico, utilizando fluxograma, sobre uma máquina genérica, que calcule o fatorial de n utilizando a fórmula $f = 1 * 2 * 3 * 4 * \dots * n$

O valor de n será fornecido pelo usuário, devendo ser um valor inteiro e positivo.

Por exemplo, caso o valor fornecido pelo usuário para n seja 5, o programa deverá apresentar como resposta o valor 120, ou seja, $1 * 2 * 3 * 4 * 5$. Caso o usuário forneça um valor inválido para n , o programa deverá apresentar uma mensagem de erro

9)

Desenvolver um programa monolítico, utilizando fluxograma, sobre uma máquina genérica, que estime o valor da constante matemática e utilizando a fórmula

$$e = 1/0! + 1/1! + 1/2! + 1/3! + \dots + 1/n!$$

O valor de n será fornecido pelo usuário, devendo ser um valor inteiro e positivo.

Por exemplo, caso o valor fornecido pelo usuário para n seja 5, o programa deverá apresentar como resposta o valor 2.72, ou seja, $1/0! + 1/1! + 1/2! + 1/3! + 1/4! + 1/5!$.

Caso o usuário forneça um valor inválido para n , o programa deverá apresentar uma mensagem de erro.

10)

Desenvolver um programa monolítico, utilizando fluxograma, sobre uma máquina genérica, que calcule o valor de e^x utilizando a fórmula $e^x = x^0/0! + x^1/1! + x^2/2! + x^3/3! + \dots + x^n/n!$

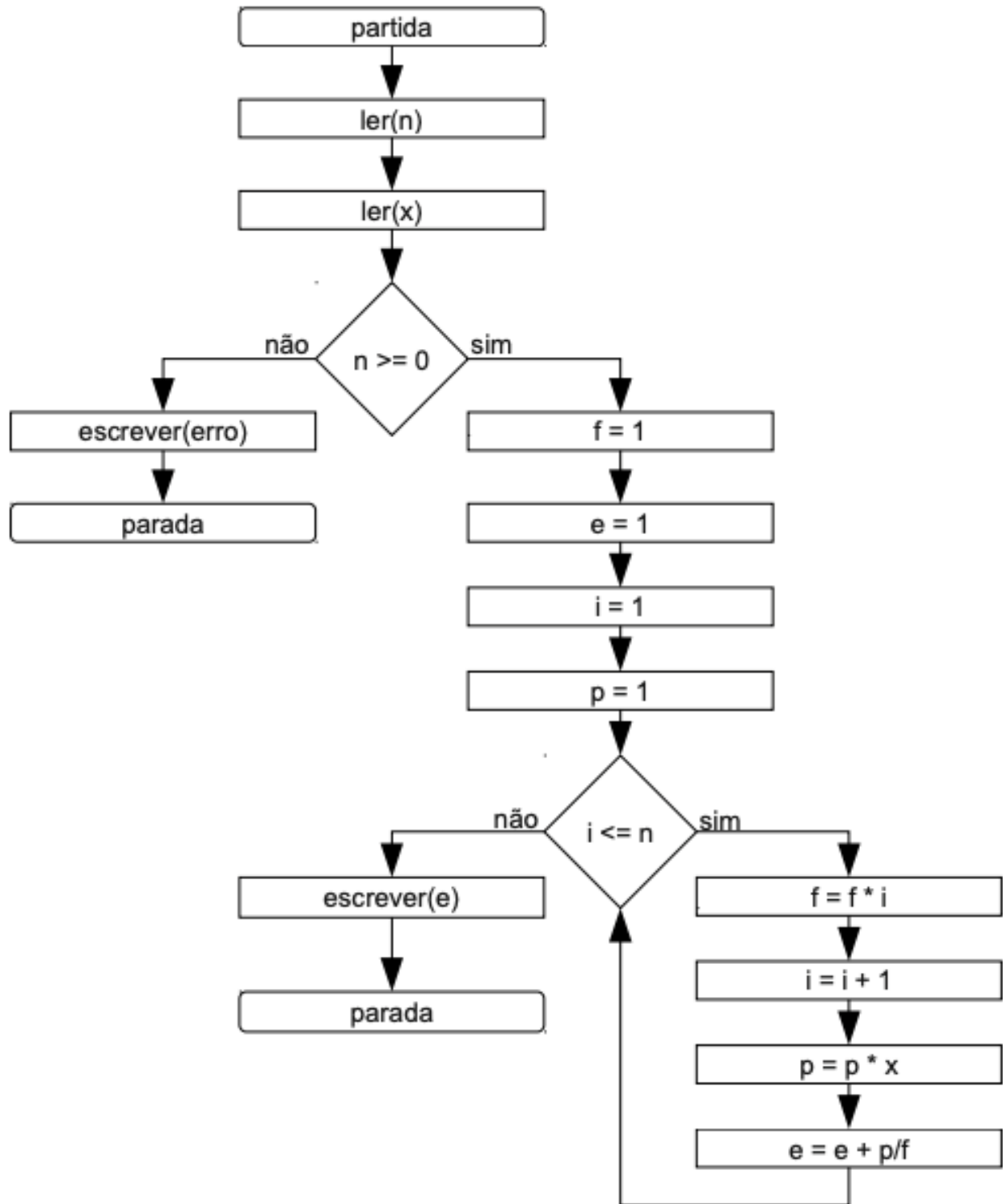
O valor de n será fornecido pelo usuário, devendo ser um valor inteiro e positivo.

O valor de x será fornecido pelo usuário, podendo ser um valor (inteiro ou real) qualquer.

Por exemplo, caso o valor fornecido pelo usuário para n seja 4 e para x seja 2, o programa deverá apresentar como resposta o valor 7, ou seja, $2^0/0! + 2^1/1! + 2^2/2! + 2^3/3! + 2^4/4!$.

Caso o usuário forneça um valor inválido para n , o programa deverá apresentar uma mensagem de erro.

Resposta: veja como foi feito o fluxograma.



11)

Desenvolver um programa monolítico, utilizando fluxograma, sobre uma máquina genérica, que apresente a série de Fibonacci. A série de Fibonacci é formada pela sequência 1, 1, 2, 3, 5, 8, 13, 21, 34, ... A série de Fibonacci é de grande importância matemática, e a lei básica é que a partir do terceiro termo, todos os termos são a soma dos dois últimos. O número de termos a serem impressos será fornecido pelo usuário, devendo ser um valor inteiro e positivo. Por exemplo, caso o número de termos a serem impressos fornecido pelo usuário seja 7, o programa deverá apresentar como resposta a sequência de valores 1, 1, 2, 3, 5, 8, 13. Caso o usuário forneça um valor inválido para o número de termos, o programa deverá apresentar uma mensagem de erro.

Programas Monolíticos com Instruções Rotuladas

12)

Desenvolver um programa monolítico, utilizando instrução rotulada, sobre uma máquina genérica, que estime o valor da constante matemática e e utilizando a fórmula $e = 1/0! + 1/1! + 1/2! + 1/3! + \dots + 1/n!$. O valor de n será fornecido pelo usuário, devendo ser um valor inteiro e positivo. Por exemplo, caso o valor fornecido pelo usuário para n seja 5, o programa deverá apresentar como resposta o valor 2.72, ou seja, $1/0! + 1/1! + 1/2! + 1/3! + 1/4! + 1/5!$. Caso o usuário forneça um valor inválido para n , o programa deverá apresentar uma mensagem de erro.

Resposta: preste atenção na solução

R1 : Faça ler(n) vá_para R2;
R2 : Se $n \leq 0$ então vá_para R3 senão vá_para R4;
R3 : Faça escrever(erro) vá_para Rx;
R4 : Faça $f = 1$ vá_para R5;
R5 : Faça $e = 1$ vá_para R6;
R6 : Faça $i = 1$ vá_para R7;
R7 : Se $i \leq n$ então vá_para R8 senão vá_para R11;
R8 : Faça $f = f * i$ vá_para R9;
R9 : Faça $i = i + 1$ vá_para R10;
R10: Faça $e = e + 1/f$ vá_para R7;
R11: Faça escrever(e) vá_para Rx;

13)

Desenvolver um programa monolítico, utilizando instrução rotulada, sobre uma máquina genérica, que calcule o valor de e^x utilizando a fórmula $e^x = x^0/0! + x^1/1! + x^2/2! + x^3/3! + \dots + x^n/n!$. O valor de n será fornecido pelo usuário, devendo ser um valor inteiro e positivo. O valor de x será fornecido pelo usuário, podendo ser um valor (inteiro ou real) qualquer. Por exemplo, caso o valor

fornecido pelo usuário para n seja 4 e para x seja 2, o programa deverá apresentar como resposta o valor 7, ou seja, $2^0/0! + 2^1/1! + 2^2/2! + 2^3/3! + 2^4/4!$. Caso o usuário forneça um valor inválido para n , o programa deverá apresentar uma mensagem de erro.

14)

Desenvolver um programa monolítico, utilizando instrução rotulada, sobre uma máquina genérica, que calcule o valor da série infinita $S = 1 + 1/2 + 1/4 + 1/6 + 1/8 + 1/10 + \dots$. O número de termos será fornecido pelo usuário, devendo ser um valor inteiro e positivo. Por exemplo, caso o número de termos fornecido pelo usuário seja 5, o programa deverá apresentar como resposta o valor 2.04, ou seja, $1 + 1/2 + 1/4 + 1/6 + 1/8$. Caso o usuário forneça um valor inválido para o número de termos, o programa deverá apresentar uma mensagem de erro.

15)

Desenvolver um programa monolítico, utilizando instrução rotulada, sobre uma máquina genérica, que apresente a série de Fibonacci. A série de Fibonacci é formada pela sequência 1, 1, 2, 3, 5, 8, 13, 21, 34, ... A série de Fibonacci é de grande importância matemática, e a lei básica é que a partir do terceiro termo, todos os termos são a soma dos dois últimos. O número de termos a serem impressos será fornecido pelo usuário, devendo ser um valor inteiro e positivo. Por exemplo, caso o número de termos a serem impressos fornecido pelo usuário seja 7, o programa deverá apresentar como resposta a sequência de valores 1, 1, 2, 3, 5, 8, 13. Caso o usuário forneça um valor inválido para o número de termos, o programa deverá apresentar uma mensagem de erro.

Programas Iterativos

16)

Desenvolver um programa iterativo, sobre uma máquina genérica, que estime o valor da constante matemática $e = 1/0! + 1/1! + 1/2! + 1/3! + \dots + 1/n!$. O valor de n será fornecido pelo usuário, devendo ser um valor inteiro e positivo. Por exemplo, caso o valor fornecido pelo usuário para n seja 5, o programa deverá apresentar como resposta o valor 2.72, ou seja, $1/0! + 1/1! + 1/2! + 1/3! + 1/4! + 1/5!$. Caso o usuário forneça um valor inválido para n , o programa deverá apresentar uma mensagem de erro.

Resposta: Preste atenção na solução.

```

programa
  ler(n);
  se (n >= 0) então
    fatorial = 1;
    e = 1;
    termo = 1;
    enquanto (termo <= n) faça
      fatorial = fatorial * termo;
      e = e + 1 / fatorial;
      termo = termo + 1;
    fim enquanto;
    escrever(e);
  senão
    escrever(erro);
  fim se;
fim programa.

```

17)

Desenvolver um programa iterativo, sobre uma máquina genérica, que calcule o valor da série infinita $S = 1 + 1/2 + 1/4 + 1/6 + 1/8 + 1/10 + \dots$. O número de termos será fornecido pelo usuário, devendo ser um valor inteiro e positivo. Por exemplo, caso o número de termos fornecido pelo usuário seja 5, o programa deverá apresentar como resposta o valor 2.04, ou seja, $1 + 1/2 + 1/4 + 1/6 + 1/8$. Caso o usuário forneça um valor inválido para o número de termos, o programa deverá apresentar uma mensagem de erro.

18)

Desenvolver um programa iterativo, sobre uma máquina genérica, que apresente a somatória dos termos da série de Fibonacci. A série de Fibonacci é formada pela sequência 1, 1, 2, 3, 5, 8, 13, 21, 34, ... A série de Fibonacci é de grande importância matemática, e a lei básica é que a partir do terceiro termo, todos os termos são a soma dos dois últimos. O número de termos será fornecido pelo usuário, devendo ser um valor inteiro e positivo. Por exemplo, caso o número de termos fornecido pelo usuário seja 7, o programa deverá apresentar como resposta o valor 33, ou seja, $1 + 1 + 2 + 3 + 5 + 8 + 13$. Caso o usuário forneça um valor inválido para o número de termos, o programa deverá apresentar uma mensagem de erro.

19)

Desenvolver um programa iterativo, sobre uma máquina genérica, que apresente a somatória de $2k$, sendo $k = 1, 2, \dots, n$. O número de termos será fornecido pelo usuário, devendo ser um valor inteiro e positivo. Por exemplo, caso o número de termos fornecido pelo usuário seja 5, o programa deverá apresentar como resposta o valor 30, ou seja, $2 * 1 + 2 * 2 + 2 * 3 + 2 * 4 + 2 * 5$. Caso o usuário

forneça um valor inválido para o número de termos, o programa deverá apresentar uma mensagem de erro.

Programas Recursivos

20)

Desenvolver um programa recursivo, sobre uma máquina genérica, que estime o valor da constante matemática $e = 1/0! + 1/1! + 1/2! + 1/3! + \dots + 1/n!$. O valor de n será fornecido pelo usuário, devendo ser um valor inteiro e positivo. Por exemplo, caso o valor fornecido pelo usuário para n seja 5, o programa deverá apresentar como resposta o valor 2.72, ou seja, $1/0! + 1/1! + 1/2! + 1/3! + 1/4! + 1/5!$. Caso o usuário forneça um valor inválido para n , o programa deverá apresentar uma mensagem de erro.

Reposta: esta resposta é só um guia mas o formato da escrita do programa recursivo definido no início é diferente desse. Por ser feito um primeiro rascunho assim e depois fazer conforme o exemplo dado no início.

Reposta: Não usar este formato de recursão.

```
função fatorial(n)
  se (n > 0)
    então retornar n * fatorial(n - 1);
    senão retornar 1;
  fim se;
fim função;

função constante(n)
  se (n > 1)
    então retornar 1 / fatorial(n - 1) + constante(n - 1);
    senão retornar 1;
  fim se;
fim função;

programa
  ler(n);
  se(n > 0) então
    e = constante(n);
    escrever(e);
  senão
    escrever(erro);
  fim se;
fim programa;
```

20-b) Converter a solução acima para a notação formal de programas recursivos, conforme o exemplo abaixo:

Usar este formato de recursão:

Converter a solução acima para a notação formal de programas recursivos, conforme o exemplo abaixo:

Exemplo 2.6 **Programa Recursivo.**

P é $R;Z$ onde
 $R \text{ def } F;(\text{se } T \text{ então } R \text{ senão } G;S),$
 $S \text{ def } (\text{se } T \text{ então } \vee \text{ senão } F;R)$
 $Z \text{ def } (\text{se } T \text{ então } G \text{ senão } F)$

figura 2.6 - Programa recursivo

21)

Desenvolver um programa recursivo, sobre uma máquina genérica, que calcule o valor da série infinita $S = 1 + 1/2 + 1/4 + 1/6 + 1/8 + 1/10 + \dots$. O número de termos será fornecido pelo usuário, devendo ser um valor inteiro e positivo. Por exemplo, caso o número de termos fornecido pelo usuário seja 5, o programa deverá apresentar como resposta o valor 2.04, ou seja, $1 + 1/2 + 1/4 + 1/6 + 1/8$. Caso o usuário forneça um valor inválido para o número de termos, o programa deverá apresentar uma mensagem de erro.

Fazer a solução usando a notação usual conforme a resposta do exercício 20 e conforme a notação formal dada pela figura acima.

Máquinas, Computações e Funções Computadas

22)

Desenvolver um programa monolítico, utilizando instruções rotuladas, sobre a máquina 2_REG, que implemente a função $B = A$. Apresentar a computação e a função computada para as entradas 3 e 4.

Resposta: Preste atenção na solução.

R1: Se a_zero então vá_para Rx senão vá_para R2;

R2: Faça subtrair_a vá_para R3;

R3: Faça adicionar_b vá_para R1;

Computação para o valor 3:

(R1, (3, 0))

(R2, (3, 0))

(R3, (2, 0))

(R1, (2, 1))

(R2, (2, 1))

(R3, (1, 1))

(R1, (1, 2))

(R2, (1, 2))

(R3, (0, 2))

(R1, (0, 3))

(Rx, (0, 3)) : 3 -> 3

23)

Desenvolver um programa monolítico, utilizando instruções rotuladas, sobre a máquina 2_REG, que implemente a função $B = A + 2$. Apresentar a computação e a função computada para as entradas 2 e 3.

$B = A + 2$

1: Se zero_a então vp 4 senao vp 2

2: sub_a vp 3

3: add_b vp 1

4: add_b vp 5

5: add_b vp 100

```

R1: Se a_zero então vá_para R4 senão vá_para R2;
R2: Faça subtrair_a vá_para R3;
R3: Faça adicionar_b vá_para R1;
R4: Faça adicionar_b vá_para R5;
R5: Faça adicionar_b vá_para Rx;

```

```

(R1, (2, 0))
(R2, (2, 0))
(R3, (1, 0))
(R1, (1, 1))
(R2, (1, 1))
(R3, (0, 1))
(R1, (0, 2))
(R4, (0, 2))
(R5, (0, 3))
(Rx, (0, 4))

```

<TCO_02, 2_REG> : 2 -> 4

```

(R1, (3, 0))
(R2, (3, 0))
(R3, (2, 0))
(R1, (2, 1))
(R2, (2, 1))
(R3, (1, 1))
(R1, (1, 2))
(R2, (1, 2))
(R3, (0, 2))
(R1, (0, 3))
(R4, (0, 3))
(R5, (0, 4))
(Rx, (0, 5))

```

<TCO_02, 2_REG> : 3 -> 5

24)

Desenvolver um programa monolítico, utilizando instruções rotuladas, sobre a máquina 2_REG, que implemente a função $B = A - 2$. Apresentar a computação e a função computada para as entradas 4 e 5.

```

R1: Faça subtrair_a vá_para R2;
R2: Faça subtrair_a vá_para R3;
R3: Se a_zero então vá_para Rx senão vá_para R4;
R4: Faça subtrair_a vá_para R5;
R5: Faça adicionar_b vá_para R3;

```

```

(R1, (4, 0))
(R2, (3, 0))
(R3, (2, 0))
(R4, (2, 0))
(R5, (1, 0))
(R3, (1, 1))
(R4, (1, 1))
(R5, (0, 1))
(R3, (0, 2))
(Rx, (0, 2))

```

<TCO_03, 2_REG> : 4 -> 2

```

(R1, (5, 0))
(R2, (4, 0))
(R3, (3, 0))
(R4, (3, 0))
(R5, (2, 0))
(R3, (2, 1))
(R4, (2, 1))
(R5, (1, 1))
(R3, (1, 2))
(R4, (1, 2))
(R5, (0, 2))
(R3, (0, 3))
(Rx, (0, 3))

```

<TCO_03, 2_REG> : 5 -> 3

25)

Desenvolver um programa monolítico, utilizando instruções rotuladas, sobre a máquina 2_REG, que implemente a função $B = A * 3$. Apresentar a computação e a função computada para as entradas 2 e 3.

1: Se zero_a então vp 100 senao vp 2
2: sub_a vp 3
3: add_b vp 4
4: add_b vp 5
5: add_b vp 1

$B = A * 3$

Para cada subtração faz 3 adicoes $B = A * 3$

Para cada subtração faz 3 adicoes.

.

```

R1: Se a_zero então vá_para Rx senão vá_para R2;
R2: Faça subtrair_a vá_para R3;
R3: Faça adicionar_b vá_para R4;
R4: Faça adicionar_b vá_para R5;
R5: Faça adicionar_b vá_para R1;

```

```

(R1, (2, 0))
(R2, (2, 0))
(R3, (1, 0))
(R4, (1, 1))
(R5, (1, 2))
(R1, (1, 3))
(R2, (1, 3))
(R3, (0, 3))
(R4, (0, 4))
(R5, (0, 5))
(R1, (0, 6))
(Rx, (0, 6))

```

<TCO_04, 2_REG> : 2 -> 6

```

(R1, (3, 0))
(R2, (3, 0))
(R3, (2, 0))
(R4, (2, 1))
(R5, (2, 2))
(R1, (2, 3))
(R2, (2, 3))
(R3, (1, 3))
(R4, (1, 4))
(R5, (1, 5))
(R1, (1, 6))
(R2, (1, 6))
(R3, (0, 6))
(R4, (0, 7))
(R5, (0, 8))
(R1, (0, 9))
(Rx, (0, 9))

```

<TCO_04, 2_REG> : 3 -> 9

26)

Desenvolver um programa monolítico, utilizando instruções rotuladas, sobre a máquina 2_REG, que implemente a função $B = A / 3$. Apresentar a computação e a função computada para as entradas 5 e 6.

$B = A / 3$.

Fazer 3 subtracoes e 1 adicao

```

1: sub_a vp 2
2: sub_a vp 3
3: Se zero_a vp 100 senao vp 4
4: sub_a vp 5
5: add_b vp 1

```

OU

```

1: Se zero_a vp 100 senao vp 2
2: sub_a vp 3
3: sub_a vp 4
4: Se zero_a vp 100 senao vp 5
5: sub_a vp 6
6: add_b vp 1

```

$B = A / 3$.

Fazer 3 subtracoes e 1 adicao

```
R1: Faça subtrair_a vá_para R2;  
R2: Faça subtrair_a vá_para R3;  
R3: Se a_zero então vá_para Rx senão vá_para R4;  
R4: Faça subtrair_a vá_para R5;  
R5: Faça adicionar_b vá_para R1;
```

```
(R1, (5, 0))  
(R2, (4, 0))  
(R3, (3, 0))  
(R4, (3, 0))  
(R5, (2, 0))  
(R1, (2, 1))  
(R2, (1, 1))  
(R3, (0, 1))  
(Rx, (0, 1))
```

<TCO_05, 2_REG> : 5 -> 1

```
(R1, (6, 0))  
(R2, (5, 0))  
(R3, (4, 0))  
(R4, (4, 0))  
(R5, (3, 0))  
(R1, (3, 1))  
(R2, (2, 1))  
(R3, (1, 1))  
(R4, (1, 1))  
(R5, (0, 1))  
(R1, (0, 2))  
(R2, (0, 2))  
(R3, (0, 2))  
(Rx, (0, 2))
```

<TCO_05, 2_REG> : 6 -> 2

27)

Desenvolver um programa monolítico, utilizando instruções rotuladas, sobre a máquina 2_REG, que implemente a função $B = (A / 2) * 3$. Apresentar a computação e a função computada para as entradas 4 e 5.

```
1: sub_a vp 2  
2: Se zero_a vp 100 senao vp 3  
3: sub_a vp 4  
4: add_b vp 5  
5: add_b vp 6  
6: add_b vp 1
```

```

R1: Faça subtrair_a vá_para R2;
R2: Se a_zero então vá_para Rx senão vá_para R3;
R3: Faça subtrair_a vá_para R4;
R4: Faça adicionar_b vá_para R5;
R5: Faça adicionar_b vá_para R6;
R6: Faça adicionar_b vá_para R1;

```

```

(R1, (4, 0))
(R2, (3, 0))
(R3, (3, 0))
(R4, (2, 0))
(R5, (2, 1))
(R6, (2, 2))
(R1, (2, 3))
(R2, (1, 3))
(R3, (1, 3))
(R4, (0, 3))
(R5, (0, 4))
(R6, (0, 5))
(R1, (0, 6))
(R2, (0, 6))
(Rx, (0, 6))

```

<TCO_07, 2_REG> : 4 -> 6

```

(R1, (5, 0))
(R2, (4, 0))
(R3, (4, 0))
(R4, (3, 0))
(R5, (3, 1))
(R6, (3, 2))
(R1, (3, 3))
(R2, (2, 3))
(R3, (2, 3))
(R4, (1, 3))
(R5, (1, 4))
(R6, (1, 5))
(R1, (1, 6))
(R2, (0, 6))
(Rx, (0, 6))

```

<TCO_07, 2_REG> : 5 -> 6

28)

Desenvolver um programa monolítico, utilizando instruções rotuladas, sobre a máquina 2_REG, que implemente a função $B = A + (A \% 3)$. Apresentar a computação e a função computada para as entradas 4 e 5.

```

R01: Se a_zero então vá_para Rx senão vá_para R02;
R02: Faça subtrair_a vá_para R03;
R03: Faça adicionar_b vá_para R04;
R04: Se a_zero então vá_para R11 senão vá_para R05;
R05: Faça subtrair_a vá_para R06;
R06: Faça adicionar_b vá_para R07;
R07: Se a_zero então vá_para R10 senão vá_para R08;
R08: Faça subtrair_a vá_para R09;
R09: Faça adicionar_b vá_para R01;
R10: Faça adicionar_b vá_para R11;
R11: Faça adicionar_b vá_para Rx;

```

```

(R01, (4, 0))
(R02, (4, 0))
(R03, (3, 0))
(R04, (3, 1))
(R05, (3, 1))
(R06, (2, 1))
(R07, (2, 2))
(R08, (2, 2))
(R09, (1, 2))
(R01, (1, 3))
(R02, (1, 3))
(R03, (0, 3))
(R04, (0, 4))
(R11, (0, 4))
(Rx , (0, 5))

```

```

(R01, (5, 0))
(R02, (5, 0))
(R03, (4, 0))
(R04, (4, 1))
(R05, (4, 1))
(R06, (3, 1))
(R07, (3, 2))
(R08, (3, 2))
(R09, (2, 2))
(R01, (2, 3))
(R02, (2, 3))
(R03, (1, 3))
(R04, (1, 4))
(R05, (1, 4))
(R06, (0, 4))
(R07, (0, 5))
(R10, (0, 5))
(R11, (0, 6))
(Rx , (0, 7))

```

<TCO_10, 2_REG> : 4 -> 5

<TCO_10, 2_REG> : 5 -> 7

29)

Desenvolver um programa monolítico, utilizando instruções rotuladas, sobre a máquina 2_REG, que implemente a função $B = (A / 2) - 3$. Apresentar a computação e a função computada para as entradas 8 e 11.

15. Desenvolver um programa monolítico, utilizando instruções rotuladas, sobre a máquina 2_REG, que implemente a função $B = (A / 2) - 3$. Apresentar a computação e a função computada para as entradas 8 e 11.

```
R1: Faça subtrair_a vá_para R2;
R2: Faça subtrair_a vá_para R3;
R3: Faça subtrair_a vá_para R4;
R4: Faça subtrair_a vá_para R5;
R5: Faça subtrair_a vá_para R6;
R6: Faça subtrair_a vá_para R7;
R7: Faça subtrair_a vá_para R8;
R8: Se a_zero então vá_para Rx senão vá_para R9;
R9: Faça adicionar_b vá_para R6;
```

```
(R1, (8, 0))
(R2, (7, 0))
(R3, (6, 0))
(R4, (5, 0))
(R5, (4, 0))
(R6, (3, 0))
(R7, (2, 0))
(R8, (1, 0))
(R9, (1, 0))
(R6, (1, 1))
(R7, (0, 1))
(R8, (0, 1))
(Rx, (0, 1))
```

<TCO_15, 2_REG> : 8 -> 1

```
(R1, (11, 0))
(R2, (10, 0))
(R3, ( 9, 0))
(R4, ( 8, 0))
(R5, ( 7, 0))
(R6, ( 6, 0))
(R7, ( 5, 0))
(R8, ( 4, 0))
(R9, ( 4, 0))
(R6, ( 4, 1))
(R7, ( 3, 1))
(R8, ( 2, 1))
(R9, ( 2, 1))
(R6, ( 2, 2))
(R7, ( 1, 2))
(R8, ( 0, 2))
(Rx, ( 0, 2))
```

<TCO_15, 2_REG> : 11 -> 2

13. Desenvolver um programa monolítico, utilizando instruções rotuladas, sobre a máquina 2_REG, que implemente a função $B = A * 3 - 2$. Apresentar a computação e a função computada para as entradas 2 e 3.

R1: Se a_zero então vá_para Rx senão vá_para R2;
R2: Faça subtrair_a vá_para R3;
R3: Faça adicionar_b vá_para R4;
R4: Se a_zero então vá_para Rx senão vá_para R5;
R5: Faça subtrair_a vá_para R6;
R6: Faça adicionar_b vá_para R7;
R7: Faça adicionar_b vá_para R8;
R8: Faça adicionar_b vá_para R4;

(R1, (2, 0))
(R2, (2, 0))
(R3, (1, 0))
(R4, (1, 1))
(R5, (1, 1))
(R6, (0, 1))
(R7, (0, 2))
(R8, (0, 3))
(R4, (0, 4))
(Rx, (0, 4))

<TCO_13, 2_REG> : 2 -> 4

(R1, (3, 0))
(R2, (3, 0))
(R3, (2, 0))
(R4, (2, 1))
(R5, (2, 1))
(R6, (1, 1))
(R7, (1, 2))
(R8, (1, 3))
(R4, (1, 4))
(R5, (1, 4))
(R6, (0, 4))
(R7, (0, 5))
(R8, (0, 6))
(R4, (0, 7))
(Rx, (0, 7))

<TCO_13, 2_REG> : 3 -> 7

08. Desenvolver um programa monolítico, utilizando instruções rotuladas, sobre a máquina 2_REG, que implemente a função $B = A - 2$ caso o valor de A seja par, ou $B = A + 2$ caso o valor de A seja ímpar. Apresentar também a computação e a função computada para as entradas 5 e 6.

```
R1: Se a_zero então vá_para Rx senão vá_para R2;
R2: Faça subtrair_a vá_para R3;
R3: Se a_zero então vá_para R4 senão vá_para R7;
R4: Faça adicionar_b vá_para R5;
R5: Faça adicionar_b vá_para R6;
R6: Faça adicionar_b vá_para R1;
R7: Faça subtrair_a vá_para R8;
R8: Se a_zero então vá_para Rx senão vá_para R5;
```

```
(R1, (5, 0))
(R2, (5, 0))
(R3, (4, 0))
(R7, (4, 0))
(R8, (3, 0))
(R5, (3, 0))
(R6, (3, 1))
(R1, (3, 2))
(R2, (3, 2))
(R3, (2, 2))
(R7, (2, 2))
(R8, (1, 2))
(R5, (1, 2))
(R6, (1, 3))
(R1, (1, 4))
(R2, (1, 4))
(R3, (0, 4))
(R4, (0, 4))
(R5, (0, 5))
(R6, (0, 6))
(R1, (0, 7))
(Rx, (0, 7))
```

<TCO_08, 2_REG> : 5 -> 7

```
(R1, (6, 0))
(R2, (6, 0))
(R3, (5, 0))
(R7, (5, 0))
(R8, (4, 0))
(R5, (4, 0))
(R6, (4, 1))
(R1, (4, 2))
(R2, (4, 2))
(R3, (3, 2))
(R7, (3, 2))
(R8, (2, 2))
(R5, (2, 2))
(R6, (2, 3))
(R1, (2, 4))
(R2, (2, 4))
(R3, (1, 4))
(R7, (1, 4))
(R8, (0, 4))
(Rx, (0, 4))
```

<TCO_08, 2_REG> : 6 -> 4

-
06. Desenvolver um programa monolítico, utilizando instruções rotuladas, sobre a máquina 2_REG, que implemente a função $B = A \% 4$. Apresentar a computação e a função computada para as entradas 5 e 6.

```
R01: Se a_zero então vá_para Rx senão vá_para R02;
R02: Faça subtrair_a vá_para R03;
R03: Se a_zero então vá_para R11 senão vá_para R04;
R04: Faça subtrair_a vá_para R05;
R05: Se a_zero então vá_para R10 senão vá_para R06;
R06: Faça subtrair_a vá_para R07;
R07: Se a_zero então vá_para R09 senão vá_para R08;
R08: Faça subtrair_a vá_para R01;
R09: Faça adicionar_b vá_para R10;
R10: Faça adicionar_b vá_para R11;
R11: Faça adicionar_b vá_para Rx;
```

```
(R01, (5, 0))
(R02, (5, 0))
(R03, (4, 0))
(R04, (4, 0))
(R05, (3, 0))
(R06, (3, 0))
(R07, (2, 0))
(R08, (2, 0))
(R01, (1, 0))
(R02, (1, 0))
(R03, (0, 0))
(R11, (0, 0))
(Rx, (0, 1))
```

<TCO_06, 2_REG> : 5 -> 1

```
(R01, (6, 0))
(R02, (6, 0))
(R03, (5, 0))
(R04, (5, 0))
(R05, (4, 0))
(R06, (4, 0))
(R07, (3, 0))
(R08, (3, 0))
(R01, (2, 0))
(R02, (2, 0))
(R03, (1, 0))
(R04, (1, 0))
(R05, (0, 0))
(R10, (0, 0))
(R11, (0, 1))
(Rx, (0, 2))
```

<TCO_06, 2_REG> : 6 -> 2

-
12. Desenvolver um programa monolítico, utilizando instruções rotuladas, sobre a máquina 2_REG, que implemente a função $B = A - (A \% 4)$. Apresentar a computação e a função computada para as entradas 4 e 5.

```
R1: Faça subtrair_a vá_para R2;  
R2: Faça subtrair_a vá_para R3;  
R3: Faça subtrair_a vá_para R4;  
R4: Se a_zero então vá_para Rx senão vá_para R5;  
R5: Faça subtrair_a vá_para R6;  
R6: Faça adicionar_b vá_para R7;  
R7: Faça adicionar_b vá_para R8;  
R8: Faça adicionar_b vá_para R9;  
R9: Faça adicionar_b vá_para R1;
```

```
(R1, (4, 0))  
(R2, (3, 0))  
(R3, (2, 0))  
(R4, (1, 0))  
(R5, (1, 0))  
(R6, (0, 0))  
(R7, (0, 1))  
(R8, (0, 2))  
(R9, (0, 3))  
(R1, (0, 4))  
(R2, (0, 4))  
(R3, (0, 4))  
(R4, (0, 4))  
(Rx, (0, 4))
```

<TCO_12, 2_REG> : 4 -> 4

```
(R1, (5, 0))  
(R2, (4, 0))  
(R3, (3, 0))  
(R4, (2, 0))  
(R5, (2, 0))  
(R6, (1, 0))  
(R7, (1, 1))  
(R8, (1, 2))  
(R9, (1, 3))  
(R1, (1, 4))  
(R2, (0, 4))  
(R3, (0, 4))  
(R4, (0, 4))  
(Rx, (0, 4))
```

<TCO_12, 2_REG> : 5 -> 4

Equivalências de Programas e Máquinas

30)

Desenvolver um programa iterativo e um programa recursivo, equivalente ao programa monolítico apresentado a seguir.

Programa Monolítico – Instruções Rotuladas

```
R1: Faça F vá_para R2;  
R2: Se T1 então vá_para R1 senão vá_para R3;  
R3: Faça G vá_para R4;  
R4: Se T2 então vá_para Rx senão vá_para R1;
```

Resposta: Preste atenção na solução

Programa Iterativo

```
programa
  F;
  enquanto T1 faça
    F;
  fim enquanto;
  G;
  enquanto não T2 faça
    F;
    enquanto T1 faça
      F;
    fim enquanto;
  G;
  fim enquanto;
fim programa.
```

Programa Recursivo

```
função principal()
  loop1();
fim função;

função loop1()
  F;
  se T1 então loop1();
    senão loop2();
  fim se
fim função;

função loop2()
  G;
  se T2 então ;
    senão loop1();
  fim se
fim função;
```

31)

Desenvolver um programa iterativo e um programa recursivo, sobre uma máquina genérica, equivalentes ao programa monolítico apresentado a seguir.

Programa Monolítico – Instruções Rotuladas

R1: faça $S = 0$ vá_para R2;

R2: faça $N = 1$ vá_para R3;

R3: se $N < 100$ então vá_para R4 senão vá_para R6;

R4: faça $S = S + N$ vá_para R5;

R5: faça $N = N + 1$ vá_para R3;

R6: faça escrever(S) vá_para R7;